



Blockchain Protocol Security Analysis Report

Customer: Neo X

Date: 04/08/2025



We express our gratitude to the Neo X team for the collaborative engagement that enabled the execution of this Blockchain Protocol Security Assessment.

Neo X implements a zero-knowledge Distributed Key Generation (DKG) protocol for Anti-MEV protection using a multi-curve architecture. The protocol enables validators to collaboratively generate threshold public-private key pairs without exposing individual secret shares.

Document

Name	Blockchain Protocol Review and Security Analysis Report for Neo X
Audited By	Reshma Fareed
Approved By	Nino Lipartiia
Website	https://neo.org/
Changelog	17/07/2025 - Preliminary Report
Changelog	04/08/2025 - Final Report
Platform	Neo X
Language	Golang
Tags	Zero Knowledge Circuit, Gnark
Methodology	https://hackenio.cc/blockchain_methodology

Review Scope

Repository	https://github.com/bane-labs/zk-dkg/tree/recursion_plonk_groth16
Commit	c79294eb21c030a2dce7b67ad11e0391f350e81a

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

15	11	4	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	2
Medium	3
Low	7

Vulnerability	Severity
F-2025-11704 - Missing Subgroup Validation for Elliptic Curve Points	High
F-2025-11705 - Non-Standard ECIES KDF Implementation	High
F-2025-11702 - Array Bounds Vulnerability in variableToU8s Function	Medium
F-2025-11703 - Missing Authentication Tag Verification in AES-GCM Circuit	Medium
F-2025-11709 - Hardcoded Beacon Challenge in SRS Sealing	Medium
F-2025-11707 - Missing Nonce Reuse Protection in AES-GCM Encryption	Low
F-2025-11708 - Hash Collision Vulnerability in Multi-Batch Processing	Low
F-2025-11710 - Missing Secure File Handling and Cleanup	Low
F-2025-11711 - Missing Input Array Validation in Main DKG Proof Generation	Low
F-2025-11712 - Theoretical Side-Channel Vulnerability in AES S-Box Implementation	Low
F-2025-11713 - Ungraceful Panic Handling in AES-GCM Decryption	Low
F-2025-11714 - No Input Validation on srsSize Parameter	Low
F-2025-11715 - Non-Standard rCon Implementation in AES-256 Key Expansion	Info
F-2025-11716 - Inefficient Hash Recomputation	Info
F-2025-11717 - Hash Function Should Include Sender and Epoch Information	Info

Documentation quality

- Protocol specification and operational guides are missing.
- Integration guides and deployment documentation are insufficient for production use.
- Threat model documentation needs development to explain attack vectors and mitigation strategies.

Code quality

- The code structure remains clear and well-organized.
- Clean separation of concerns and appropriate use of industry-standard libraries demonstrate sound engineering practices.
- Input validation, error handling, and secure memory practices require substantial improvement.
- Defensive programming patterns are inconsistent throughout the codebase.

Architecture quality

- Sound multi-curve design (secp256k1, BLS12-381, BN254) follows established cryptographic principles.
- Elegant recursive proof system supports flexible batch processing with scalable validator configurations.
- Well-structured component separation enables future protocol extensions while maintaining clean abstractions.
- Industry-standard libraries (gnark, go-ethereum) provide solid cryptographic foundations.

Test coverage

- Integration tests for end-to-end DKG ceremony workflow are missing, limiting confidence in system behavior.
- Negative test cases and security-focused scenarios need development to validate edge conditions.
- Performance and stress testing remain necessary for production deployment readiness.

Table of Contents

System Overview	6
Risks	7
Findings	8
Vulnerability Details	8
F-2025-11704 - Missing Subgroup Validation For Elliptic Curve Points - High	8
F-2025-11705 - Non-Standard ECIES KDF Implementation - High	10
F-2025-11702 - Array Bounds Vulnerability In VariableToU8s Function - Medium	12
F-2025-11703 - Missing Authentication Tag Verification In AES-GCM Circuit - Medium	14
F-2025-11709 - Hardcoded Beacon Challenge In SRS Sealing - Medium	16
F-2025-11707 - Missing Nonce Reuse Protection In AES-GCM Encryption - Low	18
F-2025-11708 - Hash Collision Vulnerability In Multi-Batch Processing - Low	20
F-2025-11710 - Missing Secure File Handling And Cleanup - Low	22
F-2025-11711 - Missing Input Array Validation In Main DKG Proof Generation - Low	25
F-2025-11712 - Theoretical Side-Channel Vulnerability In AES S-Box Implementation - Low	26
F-2025-11713 - Ungraceful Panic Handling In AES-GCM Decryption - Low	28
F-2025-11714 - No Input Validation On SrsSize Parameter - Low	29
F-2025-11715 - Non-Standard RCon Implementation In AES-256 Key Expansion - Info	31
F-2025-11716 - Inefficient Hash Recomputation - Info	32
F-2025-11717 - Hash Function Should Include Sender And Epoch Information - Info	33
Disclaimers	34
Hacken Disclaimer	34
Technical Disclaimer	34
Appendix 1. Severity Definitions	35
Appendix 2. Scope	36
Components In Scope	36

System Overview

Neo X implements a zero-knowledge Distributed Key Generation (DKG) protocol for Anti-MEV protection using a multi-curve architecture. The protocol enables validators to collaboratively generate threshold public-private key pairs without exposing individual secret shares.

Architecture Components

- **secp256k1**: Validator identity keys and ECIES encryption
- **BLS12-381**: Key share commitments and polynomial commitments
- **BN254**: Zero-knowledge proof generation and verification

DKG Process:

1. Each validator generates a polynomial and computes commitments
2. Validators encrypt key shares using ECIES and distribute to peers
3. Zero-knowledge proofs verify encryption correctness without revealing shares
4. Validators aggregate received shares to compute final secret shares
5. Global public key is computed from polynomial commitments

Risks

- **Cross-Chain Integration Risks:** External blockchain bridges and cross-chain communication protocols introduce additional attack surfaces for encrypted transaction exposure, as the Neo X Anti-MEV system must interface with external systems that may have different security assumptions. Compromise of bridge infrastructure could potentially expose encrypted transaction data before decryption or enable manipulation of the transaction ordering process.
- **Smart Contract Vulnerabilities:** The Neo X DKG protocol's smart contract acts as a critical single point of failure, managing validator governance, proof verification, curve point attack prevention, PVSS validation, BLS12-381 operations, and global public key calculations. Any vulnerability in access controls, input validation, cryptographic implementations, or governance could allow attackers to inject malicious validators, bypass security checks, or corrupt key generation. Unlike typical blockchain protocols where individual node compromises have limited impact, any successful attack against this centralized DKG contract immediately undermines the cryptographic foundations of the entire Anti-MEV system. This centralization means a single contract compromise could expose all encrypted transactions and enable unlimited MEV extraction across the network.
- **Gnark Framework Soundness:** The system depends on gnark v0.13 for zero-knowledge proof generation and verification, which is a relatively recent framework with evolving constraint system implementations and circuit compilation processes. Any soundness vulnerabilities in gnark's Plonkish constraint generation, witness assignment, or proof serialization could enable the creation of false proofs that pass verification, allowing malicious validators to prove incorrect ECIES encryption statements and inject invalid key shares into the DKG process without cryptographic detection.

Vulnerability Details

[F-2025-11704](#) - Missing Subgroup Validation for Elliptic Curve Points - High

Description:**File:** `circuit/ecies.go`

The ECIES circuit validates curve membership using `AssertIsOnCurve()` but omits subgroup membership validation. This affects both secp256k1 points for validator identity and BLS12-381 points for polynomial commitments, particularly severe for BLS12-381 curves with large cofactors.

During the DKG share phase, the circuit accepts points from small-order subgroups when validators submit polynomial commitments. This compromises the VSS security model and affects pairing checks where invalid subgroup points create cryptographic weaknesses.

Exploitation Vector: A malicious validator could craft polynomial commitments using points from small-order subgroups on BLS12-381 curves during the share phase. These invalid points would pass curve membership checks but reside in subgroups with dramatically reduced order, enabling efficient discrete logarithm attacks. Through repeated DKG operations or share reconstructions, information about other validators' secret polynomials could be leaked, gradually compromising the distributed key generation security assumptions.

Assets:

- `./circuit/ecies.go` [<https://github.com/bane-labs/zk-dkg>]

Status:`Accepted`

Classification

Impact Rate: 4/5**Likelihood Rate:** 2/5**Severity:** `High`

Recommendations

Remediation:

Add subgroup validation for `BLS12-381` curve points.

Resolution:

The client's development team conducted a comprehensive cryptographic analysis demonstrating that secp256k1's prime order structure provides inherent protection against small-subgroup attacks. Based on this analysis, they implemented an optimized solution in commit `c9e0896` that eliminates redundant `bigR` and `bigFi` variables, as these values can be derived directly from the `r` and `fi` inputs.

Following our technical review and validation of their approach, we accepted the proposed solution. Additionally, we recommended relocating the subgroup validation checks from the circuit layer to the smart contract layer for improved efficiency and security.

[F-2025-11705](#) - Non-Standard ECIES KDF Implementation - High

Description:

File: `encryption/ecies.go`

The ECIES implementation uses a custom bit-expansion KDF instead of standard constructions like HKDF or ANSI-X9.63-KDF. It transforms 64-byte elliptic curve points into 512-byte arrays through bit-level operations, omitting the ephemeral public key from KDF input.

This non-standard approach introduces statistical patterns for cryptanalytic attacks and violates ECIES key-commitment requirements. During DKG share distribution when validators encrypt polynomial evaluations `f_j(i)`, the flawed KDF enables key-commitment attacks where multiple valid decryptions exist for the same ciphertext.

Exploitation Vector: A malicious actor monitoring encrypted key share transmissions could perform statistical analysis on the bit-expansion patterns to recover information about ECDH shared secrets. Additionally, the missing ephemeral public key in the KDF enables key-commitment attacks where an adversary could craft ciphertexts that decrypt to different plaintexts depending on how the KDF is computed, potentially allowing manipulation of received key shares during the DKG distribution phase.

Assets:

- `./encryption/ecies.go` [<https://github.com/bane-labs/zk-dkg>]

Status:

Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 3/5

Severity: High

Recommendations

Remediation:

Replace with standard ECIES implementation using proper KDF with both shared secret and ephemeral public key.

```
// Get x-coordinate of shared secret point S = rPub
Sx := rPub.X.Bytes()
// Serialize ephemeral public key R
Rbytes := bigR.Bytes()
// Standard KDF with both shared secret and ephemeral public key
```

```
hash := sha3.New256()
hash.Write(Sx)
hash.Write(Rbytes) // Include ephemeral public key
key := hash.Sum(nil)
// Standard ECIES Decrypt - Replace lines 104-111
// Extract x-coordinate
Sx := rPub.X.Bytes()
// Serialize ephemeral public key
Rbytes := bigR.Bytes()
// Derive key using same KDF
hash := sha3.New256()
hash.Write(Sx)
hash.Write(Rbytes)
key := hash.Sum(nil)
```

Resolution:

The recommended fix was implemented at commit [7288671](#).

[F-2025-11702](#) - Array Bounds Vulnerability in variableToU8s Function - Medium

Description:

File: circuit/ecies.go

The `variableToU8s` function contains a critical array bounds vulnerability in the ECIES circuit implementation. When processing elliptic curve points during proof generation, the function performs array reversals without proper bounds validation.

When `len(in) > len(fixedBits)`, the expression `len(in)-1-i` produces negative indices or accesses memory beyond allocated boundaries. This affects the core encryption verification pipeline, where secp256k1 points are converted for ZK circuit constraints.

```
func variableToU8s(api frontend.API, in []frontend.Variable, nbBits
int) []uint8 {
    fixedBits := make([]frontend.Variable, 2*nbBits)
    for i := 0; i < len(fixedBits); i++ {
        if i < len(in) {
            fixedBits[len(in)-1-i] = in[i] // VULNERABILITY: Array bounds viola
tion
        }
    }
}
```

Exploitation Vector: A malicious prover or corrupted witness data could trigger this vulnerability by submitting ECIES encryption proofs with oversized point representations. When the circuit processes these malformed inputs during proof generation, the array bounds violation causes immediate system crashes, enabling denial-of-service attacks against the DKG ceremony or preventing honest validators from generating valid encryption proofs.

Assets:

- ./circuit/ecies.go [<https://github.com/bane-labs/zk-dkg>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation: Add proper bounds checking before array access

Resolution: In commit [c8e7a16](#), vulnerable `variableToU8s` function was replaced with safe `bigEndianBitsToBytes` function that validates input length (must be multiple of 8), uses safe array copying and reversal operations to prevent out-of-bounds access during ECC point marshalling.

Evidences

Poc to demonstrate : `variableToU8s` function in `circuit/ecies.go` crashes when input array is larger than expected.

Reproduce:

```
package main

import (
    "fmt"
    "github.com/consensys/gnark/frontend"
)

func vulnerableVariableToU8s(in []frontend.Variable, nbBits int) {
    fixedBits := make([]frontend.Variable, 2*nbBits)
    fmt.Printf("fixedBits length: %d\n", len(fixedBits))
    fmt.Printf("input length: %d\n", len(in))

    for i := 0; i < len(fixedBits); i++ {
        if i < len(in) {
            targetIndex := len(in) - 1 - i // VULNERABLE LINE
            fmt.Printf("Accessing fixedBits[%d] with input[%d]\n", targetIndex, i)
            fixedBits[targetIndex] = in[i] // CRASH HERE
        }
    }
}

func main() {
    // Create oversized input array
    oversizedInput := make([]frontend.Variable, 100) // Large input

    fmt.Println("Testing with nbBits=10 (fixedBits length=20)")
    vulnerableVariableToU8s(oversizedInput, 10) // Will crash
}
```

Results: Program crashes with index out of bounds error when `len(in) > len(fixedBits)`

[F-2025-11703](#) - Missing Authentication Tag Verification in AES-GCM Circuit - Medium

Description:

File: `circuit/aes256_gcm.go`

The GCM circuit implementation only verifies AES-CTR encryption correctness but completely omits authentication tag verification. The `Assert` function validates the stream cipher operation but skips polynomial authentication, effectively reducing AES-GCM to plain AES-CTR.

This breaks the authenticated encryption guarantee for key share distribution in the DKG protocol, allowing acceptance of tampered ciphertext during batch encryption proof generation.

```
func (gcm *GCM256) Assert(key [32]uint8, iv [12]uint8, chunkIndex frontend.Variable, plaintext, ciphertext []uint8) {  
    // Only checks: AES(key, IV+counter) ⊕ plaintext = ciphertext  
    // MISSING: authentication tag verification  
}
```

Exploitation Vector: A malicious validator could intercept encrypted key shares during distribution and arbitrarily modify the ciphertext while maintaining correct AES-CTR encryption patterns. The compromised circuit would accept these forged ciphertexts as valid, allowing injection of manipulated key share data into the DKG process. This enables chosen-ciphertext attacks where the attacker can influence the final distributed key generation by corrupting individual validator contributions without detection.

Assets:

- `./circuit/aes256_gcm.go` [<https://github.com/bane-labs/zk-dkg>]

Status:

Accepted

Classification

Impact Rate: 5/5

Likelihood Rate: 1/5

Severity: Medium

Recommendations

Remediation:

Implement complete GCM verification including GHASH computation and authentication tag validation.

Resolution:

The client provided the following reasoning:

We double-evaluate this issue, and we think we have enough reasons to accept it. First, the AES encrypted shares will be packed inside a blockchain transaction, so its authentication and correctness is guaranteed by the transaction signature already. Second, in our DKG scheme, we don't have a secret to verify, or I mean, we can't compute a safe enough GCM tag, since all values are public and there is no threshold to attack... So we have such a conclusion and mark this accepted, to leave the responsibility of identity and correctness verification to blockchain and EVM.

GCM mode was recommended to provide authenticated encryption, ensuring both confidentiality and integrity of the encrypted key shares against tampering or corruption during transmission.

The CTR mode decision is acceptable given the client's layered security approach. The system implements integrity protection through the ZK circuit's SHA-256 hash computation of encrypted shares, which serves as the public input binding the proof to specific ciphertext data. This circuit-level integrity verification, combined with blockchain transaction signatures and inherent ECIES authentication, provides adequate security while maintaining the client's architectural preference for separating cryptographic and blockchain responsibilities.

F-2025-11709 - Hardcoded Beacon Challenge in SRS Sealing - Medium

Description:

File: `mpc/plonk.go`

The SRS sealing process uses a hardcoded beacon challenge "beacon SRS" instead of external randomness from trusted sources like Bitcoin blocks or NIST beacons. This makes the final transformation deterministic and predictable, violating standard MPC ceremony security principles.

The predictable sealing allows adversaries with ceremony knowledge to potentially compute the final SRS in advance, compromising trusted setup guarantees. This affects the entire PLONK proof system foundation and every subsequent zero-knowledge proof generated.

```
func SealPlonkSRS(inputPath string, srsSize int) (*kzg_bn254.SRS, error) {
    prev, err := ReadPlonkSRSFromFile(inputPath, srsSize)
    if err != nil {
        return nil, err
    }
    beaconChallenge := []byte("beacon SRS") // Hardcoded value
    srs := prev.Seal(beaconChallenge)
    return &srs, nil
}
```

Exploitation Vector: An adversary with knowledge of the ceremony contribution chain and the hardcoded beacon string "beacon SRS" could deterministically compute the final SRS parameters. If the attacker previously participated in the ceremony or obtained ceremony state information, this predictability could enable them to derive relationships between the final SRS and their contribution, potentially creating backdoors or trapdoors that compromise the zero-knowledge proof system's soundness guarantees.

Assets:

- `./mpc/plonk.go` [<https://github.com/bane-labs/zk-dkg>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation: Use external randomness beacon from trusted source like Bitcoin block hash or NIST beacon

Resolution: The client appropriately addressed the hard-coded beacon challenge vulnerability by requiring it as a command-line parameter by implementing it at commit [fa78402](#) , eliminating predictability that could enable pre-computation attacks.

[F-2025-11707](#) - Missing Nonce Reuse Protection in AES-GCM Encryption - Low

Description:

File: `encryption/aes.go`

The AES-GCM implementation generates random nonce but provides no mechanism to prevent nonce reuse, which would catastrophically compromise security. While generating 12-byte random nonce correctly, it lacks protection against reuse due to poor entropy, VM state replication, or system resets.

In AES-GCM, nonce reuse with the same key allows recovery of the authentication key through polynomial collision attacks and enables plaintext recovery via keystream reuse. This is critical in distributed DKG where multiple validators might experience synchronized state resets.

```
nonce := make([]byte, 12)
if _, err := io.ReadFull(rand.Reader, nonce); err != nil {
    return nil, nil, err
}
// No tracking or reuse prevention
```

Failure Scenario: During DKG operations, if system entropy is compromised or validators experience state resets (such as VM rollbacks or synchronized restarts), the same nonce could be generated multiple times with identical keys. When this occurs, an adversary monitoring network traffic can XOR the duplicate ciphertexts to eliminate the keystream and recover the XOR of plaintexts. Through cryptanalytic techniques on the resulting XOR, the attacker can reconstruct the original key share data, compromising the distributed key generation process.

Assets:

- `./encryption/aes.go` [<https://github.com/bane-labs/zk-dkg>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

Implement nonce tracking with collision detection or use deterministic nonce construction

Resolution:

The client implemented counter-based nonce generation in the commit [18cd2a4](#) using global counter in first 4 bytes (BigEndian) with remaining 8 bytes from crypto/rand. This deterministic approach prevents nonce reuse while maintaining sufficient entropy, similar to Ethereum's P2P encryption method.

[F-2025-11708](#) - Hash Collision Vulnerability in Multi-Batch Processing - Low

Description:

File: `circuit/circuit.go`

Batch hash computation uses simple concatenation without domain separation, potentially enabling hash collisions across different batch configurations. The recursive proof architecture supports batch sizes of 1, 2, and 7 encryptions, with hash computation for public input binding.

Without domain separation including batch size or proof index, `hash(A||B)` could equal `hash(C||D)` for different batch components. This enables proof substitution attacks where valid proofs for one batch configuration are accepted for completely different batches.

```
allHash = append(allHash, innerhash...) // Simple concatenation
```

Exploitation Vector: A malicious prover could carefully craft encryption inputs across different batch sizes to produce hash collisions in the concatenated proof data. By finding inputs where `hash(batch1_data||batch2_data)` equals `hash(different_batch1||different_batch2)`, the attacker could create alternative valid proofs for the same hash commitment, potentially substituting legitimate encryption proofs with malicious ones that pass verification but contain different underlying key share data.

Assets:

- `./circuit/circuit.go` [<https://github.com/bane-labs/zk-dkg>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

Add domain separation with batch size encoding in hash computation.

Resolution:

The fix was implemented at commit [abc820b](#) by adding domain separation, prepending "DKG_BATCH_HASH_V1" constant and batch size to hash computation. Implemented as hardcoded uint8 array followed by parameter count encoding to prevent cross-batch collision attacks.

F-2025-11710 - Missing Secure File Handling and Cleanup - Low

Description:

File: `mpc/plonk.go`

SRS files containing sensitive cryptographic material are created with default permissions and non-atomic write operations. During the ceremony workflow, participants read the previous SRS state and write updated data without proper file security or atomic operations.

Non-atomic writes can leave partial or corrupted SRS data if interrupted, invalidating the ceremony chain. Default permissions and a lack of secure cleanup allow unauthorized access to intermediate states and potential recovery of contribution data.

```
func ContributePlonkSRS(prevPath string, nextPath string, srsSize int) (*kzg_bn254.MpcSetup, error) {
    // ... contribution logic
    out, err := os.Create(nextPath) // Default permissions, no atomic write
    if err != nil {
        return nil, err
    }
    defer out.Close()
    _, err = p.WriteTo(out) // No verification of complete write
    if err != nil {
        return nil, err
    }
    return p, nil
}
```

System Compromise Scenario: An attacker with local file system access could read intermediate SRS files created with overly permissive permissions, potentially extracting contribution data or ceremony state information. Additionally, if the ceremony is interrupted during non-atomic write operations (due to system crashes or resource constraints), partial SRS data could remain in an inconsistent state, corrupting the ceremony chain and forcing restart of the entire trusted setup process, wasting previous participants' contributions.

Assets:

- `./mpc/plonk.go` [<https://github.com/bane-labs/zk-dkg>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation: Implement atomic file operations with restrictive permissions (0600) and secure cleanup of temporary files:

```
func ContributePlonkSRS(prevPath string, nextPath string, srsSize int) (*kzg_bn254.MpcSetup, error) {
    p, err := ReadPlonkSRSFromFile(prevPath, srsSize)
    if err != nil {
        return nil, err
    }

    p.Contribute()

    // Atomic write with secure permissions
    err = writeSecureAtomic(nextPath, p)
    if err != nil {
        return nil, err
    }

    return p, nil
}

func writeSecureAtomic(finalPath string, data *kzg_bn254.MpcSetup) error {
    // Create temporary file with restrictive permissions
    tempPath := finalPath + ".tmp"
    out, err := os.OpenFile(tempPath, os.O_CREATE|os.O_WRONLY|os.O_EXCL, 0600)
    if err != nil {
        return err
    }

    defer func() {
        out.Close()
        // Secure cleanup: overwrite and delete temp file on failure
        if err != nil {
            secureDelete(tempPath)
        }
    }()

    // Write data completely
    _, err = data.WriteTo(out)
    if err != nil {
        return err
    }

    // Ensure data is written to disk
    err = out.Sync()
    if err != nil {
        return err
    }

    err = out.Close()
    if err != nil {
        return err
    }

    // Atomic rename
    return os.Rename(tempPath, finalPath)
}

func secureDelete(path string) {
    if file, err := os.OpenFile(path, os.O_WRONLY, 0); err == nil {
        // Overwrite with random data
        stat, _ := file.Stat()
        randomData := make([]byte, stat.Size())
        rand.Read(randomData)
        file.WriteAt(randomData, 0)
        file.Sync()
        file.Close()
    }
    os.Remove(path)
}
```

Resolution:

The recommended fix was applied at commit [fb29104](#).

[F-2025-11711](#) - Missing Input Array Validation in Main DKG Proof Generation - Low

Description:

File: `zk_dkg.go`

The main `ProveMultipleKeyShareEncryption` function lacks input validation for array lengths and consistency checks. It coordinates between `BLS12-381` field processing, `secp256k1` ECIES encryption, and `BN254` circuit proof generation without validating array consistency or boundary conditions.

Mismatched array lengths cause index out-of-bounds errors during multi-curve processing, and missing validation for empty arrays or malformed elements leads to undefined behavior. This propagates through the entire proof pipeline from field conversion to circuit witness generation.

Failure Scenario: During DKG operations, if a validator submits mismatched array lengths for key shares and recipient public keys, or provides empty input arrays, the proof generation process encounters unhandled edge cases leading to system panics. This could enable denial-of-service attacks where malicious actors submit malformed inputs to crash honest validators' proof generation, disrupting the DKG ceremony or preventing legitimate participants from completing their cryptographic commitments.

Assets:

- `./zk_dkg.go` [<https://github.com/bane-labs/zk-dkg>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation: Add comprehensive input validation for all array parameters before processing

Resolution: The recommended fix was implemented at commit [ee48eb3](#).

[F-2025-11712](#) - Theoretical Side-Channel Vulnerability in AES S-Box Implementation - Low

Description:

File: `circuit/aes256.go`

The S-box implementation uses data-dependent timing through `IsZero()` operations that could leak information during witness generation. The `subw` function implements S-box lookups using linear scan with timing variations based on input values.

During witness generation on actual key share data, the timing of S-box operations could vary based on match positions in the 256-element lookup. This requires sophisticated timing analysis and local machine access, making it primarily theoretical in typical deployments.

```
func (aes *AES256) subw(sbox []uint8, a uint8) uint8 {
    for j := 0; j < 256; j++ {
        // Timing varies based on (a.Val - j) == 0
        out = aes.api.Add(out, aes.api.Mul(aes.api.IsZero(aes.api.Sub(a.Val, j)), sbox[j].Val))
    }
}
```

Side-Channel Scenario: An adversary with physical or privileged access to a validator's proof generation system could perform high-precision timing analysis during witness generation to detect timing variations in S-box operations. Through statistical analysis across multiple proof generations, this could theoretically leak information about AES S-box inputs, though such attacks require sophisticated measurement equipment and local system access, making them primarily relevant in high-security threat models.

Assets:

- `./circuit/aes256.go` [<https://github.com/bane-labs/zk-dkg>]

Status:

Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

Consider implementing bitsliced S-box or add timing randomization during witness generation.

Resolution:

Client's Team accepted the finding noting that zk-dkg is integrated directly into Geth nodes where local machine access by attackers would already compromise the entire validator security model. Theoretical timing attacks are not practically relevant in their deployment architecture.

[F-2025-11713](#) - Ungraceful Panic Handling in AES-GCM

Decryption - Low

Description:**File:** `encryption/aes.go`

AES-GCM decryption panics on invalid nonce lengths instead of returning errors, causing ungraceful failures during encrypted key share processing. When validators receive a malformed ciphertext with incorrect nonce lengths, the system crashes instead of handling errors gracefully.

This affects DKG share distribution robustness and prevents proper error logging and recovery mechanisms. Malicious participants could exploit this by sending corrupted encrypted data to cause validator crashes during decryption attempts.

Denial-of-Service Scenario: A malicious participant could deliberately send encrypted key shares with invalid nonce lengths to targeted validators during the DKG distribution phase. When honest validators attempt to decrypt these malformed ciphertexts, the system panics instead of gracefully handling the error, causing validator processes to crash and potentially disrupting the entire DKG ceremony or requiring a ceremony restart.

Assets:

- `./encryption/aes.go` [<https://github.com/bane-labs/zk-dkg>]

Status:**Fixed**

Classification

Impact Rate: 1/5**Likelihood Rate:** 3/5**Severity:** **Low**

Recommendations

Remediation: Add nonce length validation before calling `aesgcm.Open()`.**Resolution:** The recommended fix was implemented at commit [7bc50b7](#).

F-2025-11714 - No Input Validation on srsSize Parameter - Low

Description:

File: `mpc/plonk.go`

The `srsSize` parameter is accepted without validation across all MPC functions, despite documentation indicating valid ranges. The parameter controls SRS size and directly impacts memory allocation, computation time, and cryptographic security level.

Extremely large values could cause memory exhaustion or system crashes, while values outside intended ranges could compromise cryptographic assumptions. This enables resource exhaustion attacks where malicious participants specify excessive `srsSize` values during ceremony setup.

```
func InitPlonkSRS(path string, srsSize int) (*kzg_bn254.MpcSetup, error) {  
    srs := kzg_bn254.InitializeSetup(srsSize) // No validation of srsSize  
    srs.Contribute()  
    // ... rest of function  
}
```

Resource Exhaustion Scenario: A malicious ceremony participant or system administrator could specify an excessively large `srsSize` value during ceremony initialization, causing the system to attempt allocating massive amounts of memory for SRS generation. This could lead to system crashes, memory exhaustion, or denial-of-service conditions that disrupt the ceremony process. While Go's type system prevents some categories of invalid input, the lack of application-level bounds checking still allows values that could overwhelm system resources.

Assets:

- `./mpc/plonk.go` [<https://github.com/bane-labs/zk-dkg>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

- Remediation:** Add input validation to ensure srsSize is within the documented range of 1-27.
- Resolution:** The recommended fix was implemented at commit [c6b27b8](#).

[F-2025-11715](#) - Non-Standard rCon Implementation in AES-256 Key Expansion - Info

Description:

File: `circuit/aes256.go`

The AES implementation uses non-standard `rCon` constants starting with `0x8d` instead of the FIPS-197 specified values, though the first element is not accessed during key expansion.

```
rCon := uints.NewU8Array([]uint8{0x8d, 0x01, 0x02, 0x04, ...})
// Should be: {0x00, 0x01, 0x02, 0x04, ...} or start with 0x01
```

Standards Compliance Issue: The non-standard `rCon` array could create integration challenges when interfacing with other cryptographic systems or when auditing against FIPS-197 compliance requirements. While functionally equivalent due to the unused first element, the deviation may cause confusion during security reviews or when validating the implementation against established AES standards.

Assets:

- `./circuit/aes256.go` [<https://github.com/bane-labs/zk-dkg>]

Status:

Fixed

Classification

Likelihood Rate: 1/5

Severity: Info

Recommendations

Remediation: Use standard FIPS-197 `rCon` values for consistency and compliance.

Resolution: The recommended fix was implemented at commit [893088a](#).

[F-2025-11716](#) - Inefficient Hash Recomputation - Info

Description:

File: `circuit/circuit.go`

The `ComputeMultipleKeyShareEncryptionAssignment` function recomputes hash values instead of reusing existing results.

Performance Impact: The redundant hash computations create unnecessary computational overhead during proof generation, increasing resource consumption and proof generation latency. While not a security vulnerability, this inefficiency could impact system performance in high-throughput scenarios or resource-constrained environments.

Assets:

- `./circuit/circuit.go` [<https://github.com/bane-labs/zk-dkg>]

Status:

Fixed

Classification

Likelihood Rate: 4/5

Severity: Info

Recommendations

Remediation: Cache and reuse hash computation results where possible.

Resolution: The fix was implemented at commit [cd50a81](#).

[F-2025-11717](#) - Hash Function Should Include Sender and Epoch Information - Info

Description:

File: `circuit/circuit.go`

The `computeSumHash` function could be strengthened by binding additional context information like sender identity and epoch data.

Context Binding Enhancement: The absence of sender and epoch information in hash computation could theoretically allow proof reuse across different contexts, though the current implementation provides adequate security for the intended DKG use case. Additional binding would provide defense-in-depth against potential future attack vectors.

Assets:

- `./circuit/circuit.go` [<https://github.com/bane-labs/zk-dkg>]

Status:

Accepted

Classification

Likelihood Rate: 1/5

Severity: Info

Recommendations

Remediation: Consider including sender public key and epoch information in hash computation for stronger proof binding.

Resolution: Commitment uniqueness validation will be implemented in the smart contract post-audit as contracts are out of scope.

Disclaimers

Hacken Disclaimer

The blockchain protocol given for audit has been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in the protocol source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other protocol statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of the blockchain protocol.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

Technical Disclaimer

Blockchain protocols are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the protocol can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited blockchain protocol.

Appendix 1. Severity Definitions

Severity	Description
Critical	Vulnerabilities that can lead to a complete breakdown of the blockchain network's security, privacy, integrity, or availability fall under this category. They can disrupt the consensus mechanism, enabling a malicious entity to take control of the majority of nodes or facilitate 51% attacks. In addition, issues that could lead to widespread crashing of nodes, leading to a complete breakdown or significant halt of the network, are also considered critical along with issues that can lead to a massive theft of assets. Immediate attention and mitigation are required.
High	High severity vulnerabilities are those that do not immediately risk the complete security or integrity of the network but can cause substantial harm. These are issues that could cause the crashing of several nodes, leading to temporary disruption of the network, or could manipulate the consensus mechanism to a certain extent, but not enough to execute a 51% attack. Partial breaches of privacy, unauthorized but limited access to sensitive information, and affecting the reliable execution of smart contracts also fall under this category.
Medium	Medium severity vulnerabilities could negatively affect the blockchain protocol but are usually not capable of causing catastrophic damage. These could include vulnerabilities that allow minor breaches of user privacy, can slow down transaction processing, or can lead to relatively small financial losses. It may be possible to exploit these vulnerabilities under specific circumstances, or they may require a high level of access to exploit effectively.
Low	Low severity vulnerabilities are minor flaws in the blockchain protocol that might not have a direct impact on security but could cause minor inefficiencies in transaction processing or slight delays in block propagation. They might include vulnerabilities that allow attackers to cause nuisance-level disruptions or are only exploitable under extremely rare and specific conditions. These vulnerabilities should be corrected but do not represent an immediate threat to the system.

Appendix 2. Scope

The scope of the project includes the following components from the provided repository:

Scope Details	
Repository	https://github.com/bane-labs/zk-dkg/tree/recursion_plonk_groth16
Commit	c79294eb21c030a2dce7b67ad11e0391f350e81a

Components in Scope

This audit covers all production code in:

- `circuit/` - Zero-knowledge circuit implementations
- `encryption/` - Cryptographic primitives (AES-GCM, ECIES)
- `helper/` - Proof generation and utilities
- `mpc/` - Multi-party computation setup
- `zk_dkg.go` - Main DKG entry point

Test files and documentation were excluded from scope.